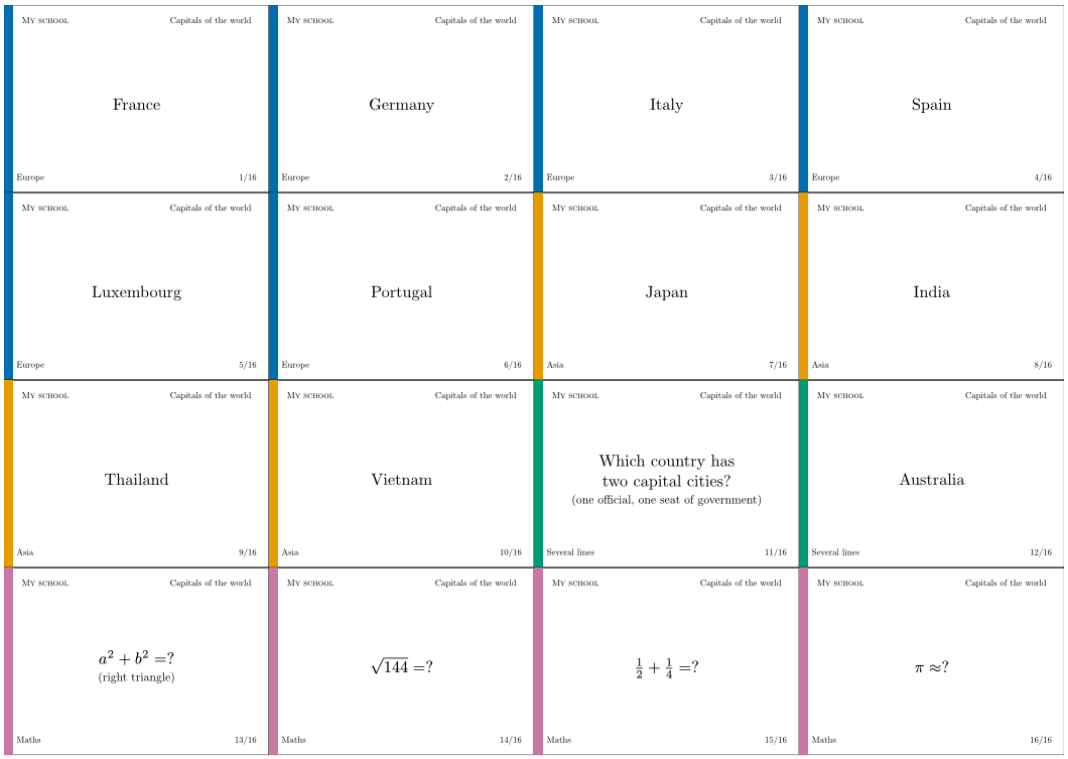
flipside

Double-sided flash cards that line up when printed

Manual · version 0.2.0

Frank Fasbender · ffasbend@gmail.com

github.com/ffasbend/flipside



flipside lays out flash cards for double-sided printing. The odd pages hold the questions with cutting lines, the even pages the answers, placed so that each answer lands exactly behind its question. The same file also gives a **list** to check every card before printing, and a **calibration sheet** to make up for printers that shift the back side.

Contents

1 Getting started ... 3
1.1 What you get ... 3
1.2 Installing ... 3
1.3 A first deck ... 4
2 Writing cards ... 5
2.1 Several lines, small print, lists ... 5
2.2 Themes and other fields ... 6
2.3 Cards made by a function ... 6
2.4 Cards from a CSV file ... 7
3 Both directions, items and hints ... 8
3.1 Directions ... 9
3.2 Several items: `all` and `any` ... 9
3.3 Shown only as question, or only as answer ... 9
3.4 For the MyCards app ... 9
4 Header and footer ... 10
5 Language, fonts and size ... 12
6 Text that is too large ... 13
7 Coloured stripes per theme ... 14
8 Card sizes and paper ... 15
8.1 The sizes ... 15
8.2 Paper ... 15
8.3 Upright cards ... 16
8.4 A size of your own ... 16
9 Printing ... 17
9.1 Duplex settings ... 17
9.2 Calibration: when the back side is shifted ... 17
9.3 Reprinting some cards ... 18
10 The list: checking the cards ... 19
11 Command line ... 20
12 Reference ... 21
13 Examples ... 23
13.1 Vocabulary from a CSV file ... 23
13.2 Maths ... 24
13.3 Business-card sheets ... 25
13.4 US index cards ... 25
13.5 Upright cards of your own size ... 26
14 Questions and answers ... 27

1 Getting started

1.1 What you get

A deck is a list of cards, each with a question side `q` and an answer side `a`. flipside puts as many cards as fit on a sheet (16 for the default DIN A8 size on A4), the questions on one page and the answers on the next. When the sheet is printed double-sided and cut along the lines, every card has its question on the front and its answer on the back.

France	Germany	Italy	Spain

question page (odd page)

Madrid	Rome	Berlin	Paris

answer page (even page): the columns are mirrored

The answer page is the question page **mirrored left to right**: the first card of a row is at the right on the answer page. That is what makes the answer land behind its question when the sheet is turned over like a book page.

1.2 Installing

Once the package is on Typst Universe, nothing has to be installed: the import below downloads it the first time you compile.

```
#import "@preview/flipside:0.2.0": *
```

To start a new deck from the template:

```
typst init @preview/flipside:0.2.0 my-cards
```

> **Before publication, or to use your own copy:** put the package folder into Typst's local package folder (macOS: `~/Library/Application Support/typst/packages/local/flipside/0.2.0`, Linux: `~/.local/share/typst/packages/local/flipside/0.2.0`) and import `@local/flipside:0.2.0` instead. A relative import such as `#import "flipside/lib.typ": *` works too, and also in the Typst web app.

1.3 A first deck

```
#import "@preview/flipside:0.2.0": *

#flipside((
  (q: [France], a: [Paris]),
  (q: [Germany], a: [Berlin]),
  (q: [Italy], a: [Rome]),
  (q: [Spain], a: [Madrid]),
))
```

Compile it with `typst compile cards.typ`, or in the Typst web app or your editor. The result is the sheet shown above: page 1 with the questions, page 2 with the answers. Print **all pages double-sided** (see Section 9).

2 Writing cards

2.1 Several lines, small print, lists

Write one printed line per source line and end every line but the last with `\`. Everything Typst can do works inside a card: maths, bold, lists, images.

```
#import "@preview/flipside:0.2.0": *

#flipside((
  (
    q: [
      Which country has \
      two capital cities?
    ],
    a: [
      Bolivia \
      #small[(Sucre and La Paz)]
    ],
  ),
  (
    q: [$a^2 + b^2 = ?$ \ #tiny[(right triangle)]],
    a: [$c^2$, the square \ of the hypotenuse],
  ),
  (
    q: [Name the three \ states of matter.],
    a: block-left[
      - solid
      - liquid
      - gas
    ],
  ),
  (
    q: [What is a prime number?],
    a: block-left[
      A natural number greater than 1 that has no divisors other than 1 and itself.
    ],
  ),
))
```

Which country has two capital cities?	$a^2 + b^2 = ?$ (right triangle)	Name the three states of matter.	What is a prime number?

question page (odd page)

A natural number greater than 1 that has no divisors other than 1 and itself.	• solid • liquid • gas	c^2, the square of the hypotenuse	Bolivia (Sucre and La Paz)

answer page (even page): the columns are mirrored

The helpers `small[…]` (72 %) and `tiny[…]` (60 %) make text smaller, `block-left[…]` aligns a longer text left and justifies it (cards are centred by default).

> A line starting with `1.` or `-` becomes a numbered or bulleted list in Typst. To print "1." as plain text at the start of a line, write `1\.`.

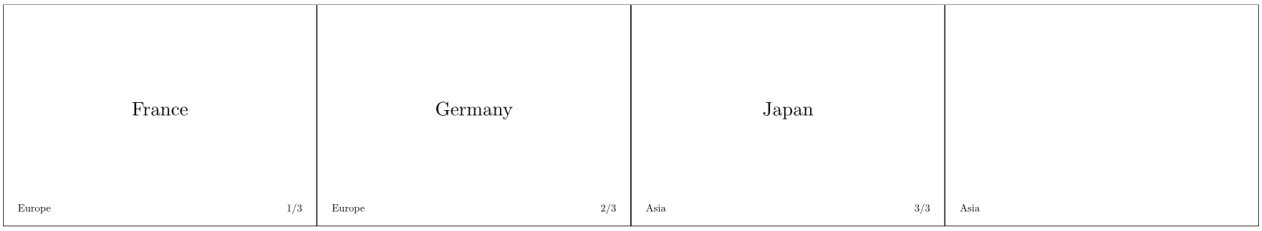
2.2 Themes and other fields

An entry **without** `q` and `a` is no card and takes no place on the sheet. Its fields are given to every card after it, until another entry changes them. Use it for themes, chapters, levels, anything the header and footer should show (Section 4).

```
#import "@preview/flipside:0.2.0": *

#let number(c) = if c.number != none [#c.number/#c.total]

#flipside(
  footer: (left: c => c.theme, right: number),
  (
    (theme: "Europe"),      // no card: the theme of the cards below
    (q: [France], a: [Paris]),
    (q: [Germany], a: [Berlin]),
    (theme: "Asia"),
    (q: [Japan], a: [Tokyo]),
    (q: [], a: []),         // a blank card
  ),
)
```



`c.theme` in the footer; `(q: [], a: [])` is a blank card: an empty place on the sheet, without number

Every card also gets `number` (1, 2, …; `none` for blank cards) and `total` (the number of cards without the blank ones).

2.3 Cards made by a function

When many cards look alike, let a function make them:

```
#let element(symbol, name, number) = (
  q: text(size: 2.4em, weight: "bold", symbol),
  a: [#name \ #small[atomic number #number]],
)

#flipside((
  element("Li", "Lithium", 3),
  element("Na", "Sodium", 11),
))
```

The same goes for questions with a common pattern:

```
#let digit(d) = [Which place has the digit #d \ in the number 1'234'056.789?]
#flipside((
  (q: digit(6), a: [units]),
  (q: digit(5), a: [tens]),
))
```

> Typst turns `'` after a digit into a prime (′). To get a typographic apostrophe as thousands separator, write `’` (or `\u{2019}`) in that number.

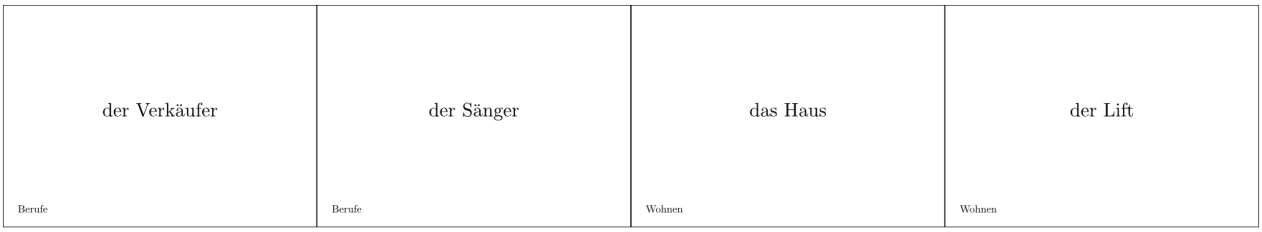
2.4 Cards from a CSV file

A word list from Excel, LibreOffice or Google Sheets, saved as CSV, can be used directly: Typst reads CSV files itself.

```
q,a,theme
der Verkäufer,"le vendeur
la vendeuse",Berufe
der Sänger,"le chanteur
la chanteuse",
das Haus,la maison,Wohnen
der Lift,l'ascenseur,
```

```
#import "@preview/flipside:0.2.0": *

#flipside(
  lang: (q: "de", a: "fr"),
  footer: (left: c => c.theme),
  csv("words.csv", row-type: dictionary),
)
```



The answer side (page 2) shows “le vendeur / la vendeuse” on two lines.

- Columns `q` and `a` are the two sides; every other column (here `theme`) becomes a field of the card.
- An **empty cell** keeps the value of the rows above, so the theme is written only once.
- A row with **no question and no answer** but other cells only sets those fields, like `(theme: …)` above.
- A **line break inside a cell** is a line break on the card.
- Without a header row, `csv("words.csv")` gives rows of (question, answer).
- Excel often uses `;` as separator: `csv("words.csv", delimiter: ";")`.
- Cells are plain text. With `markup: true` they are read as Typst markup, so `$x^2$`, `*bold*` or `_italic_` work.
- CSV cards and written cards can be mixed:

```
#flipside(
  csv("words.csv", row-type: dictionary)
    + ((theme: "Extra"), (q: [Hallo], a: [Salut]))
)
```

3 Both directions, items and hints

A card can be studied both ways: German → French and French → German. Each side then has two **roles**: as the **question** it is what is asked, as the **answer** what is checked. flipside prints every part of a side as it should be in that role, so one list gives correct cards both ways. The MyCards app reads the same marks.

```
#import "@preview/flipside:0.2.0": *

#flipside(
  lang: (q: "de", a: "fr"),
  dir: "both",                    // every card both ways
  footer: (left: c => if c.direction == "forward" [de → fr] else [fr → de]),
  (
    (q: [laut [? (m/f)]], a: [bruyant, bruyante]),
    (q: [der Lift, [? ein Lift]], a: [l'ascenseur, [! un ascenseur]]),
    (q: [der Kühlschrank], a: any[le réfrigérateur][le frigo]),
    (
      q: [die Möbel #q-note[Plural]],
      a: all[les meubles][le mobilier][l'ameublement],
      dir: "forward",             // this card only one way
    ),
  ),
)
```

laut (m/f) de → fr	bruyant, bruyante fr → de	der Lift, ein Lift de → fr	l'ascenseur fr → de
der Kühlschrank de → fr	le réfrigérateur fr → de	le frigo fr → de	die Möbel Plural (3) de → fr

question page (odd page)

der Lift	l'ascenseur, un ascenseur	laut	bruyant, bruyante
les meubles le mobilier l'ameublement	der Kühlschrank	der Kühlschrank	le réfrigérateur / le frigo

answer page (even page): the columns are mirrored

3.1 Directions

Write	Cards
`dir: "forward"` (default)	side `q` is asked, `a` is the answer
`dir: "reverse"`	side `a` is asked, `q` is the answer
`dir: "both"`	every card twice, once each way

`dir` is given to `#flipside(…)` for the deck, and in a card for that card only; `(dir: "forward"),` as an entry of its own holds for the cards below it, like a theme. `--input dir=both` chooses it when compiling. Every printed card has `c.direction` (`"forward"` or `"reverse"`) for the slots, as in the footer above; `c.front-lang` and `c.back-lang` are its languages.

3.2 Several items: `all` and `any`

Write	Side is the question	Side is the answer
`all[…][…]`	one card, items together	**all** needed; on lines of their own
`any[…][…]`	**one card per item**	**any one** is right; "… / …"
`any(a: "all")[…][…]`	one card per item	all needed
`all(a: "any")[…][…]`	one card, items together	any one is right

- When more than 2 items are needed, the question says how many: "(3)". `count: n => [(#n réponses)]` changes the text.
- A comma never splits anything: `[bruyant, bruyante]` is **one** answer, the normal way to write forms that belong together.

3.3 Shown only as question, or only as answer

Write	Side is the question	Side is the answer
`[? …]` (plain text)	shown, without brackets	–
`[! …]` (plain text)	–	shown, without brackets
`#q-only[…]`	shown	–
`#a-only[…]`	–	shown
`#note[…]`	shown, on its own line	shown, on its own line
`#q-note[…]`	shown, on its own line	–
`#a-note[…]`	–	shown, on its own line

- `[? …]` and `[! …]` are typed as normal text, as in the text files of the MyCards app; the text inside must be plain. For maths, bold or a picture use `#q-only[…]` / `#a-only[…]`. Brackets in maths (`$[0, 1]$`) are never taken for them.
- A comma or space before a hidden `[? …]` / `[! …]` goes with it: "der Lift, [? ein Lift]" as the answer is "der Lift".

3.4 For the MyCards app

The app reads flipside decks with `--input layout=report`: the cards as written (with `dir`, themes and all marks) as metadata `<flipside-cards>`, and every picture on a page of its own. Nothing has to be changed in the deck.

4 Header and footer

Each side of a card has a header and a footer with three places, called **slots**: `left`, `center` and `right`.

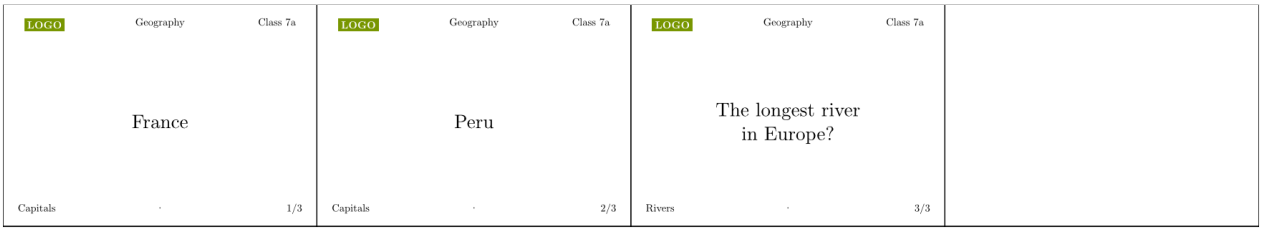
Parameter	Side	Example
`header`	question side, top	`(left: logo, right: [Class 7a])`
`footer`	question side, bottom	`(left: c => c.theme)`
`back-header`	answer side, top	`(right: [Answers])`
`back-footer`	answer side, bottom	`(right: number)`

A slot is content (the same on every card) or a **function** that gets the card and returns content. The card has `q`, `a`, `number`, `total` and the fields of the entries above it.

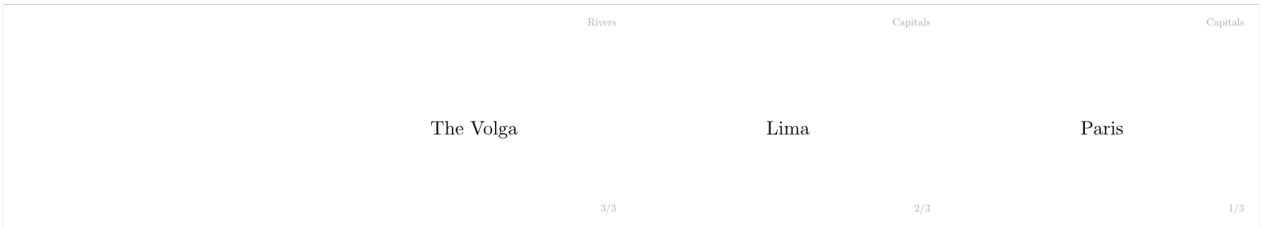
```
#import "@preview/flipside:0.2.0": *

#let number(c) = if c.number != none [#c.number/#c.total]
#let logo = box(fill: rgb("#7a9a01"), inset: 2pt, text(fill: white)[*LOGO*])

#flipside(
  header: (left: logo, center: [Geography], right: [Class 7a]),
  footer: (left: c => c.theme, center: [·], right: number),
  back-header: (right: c => text(fill: gray)[#c.theme]),
  back-footer: (right: c => text(fill: gray, number(c))),
  (
    (theme: "Capitals"),
    (q: [France], a: [Paris]),
    (q: [Peru], a: [Lima]),
    (theme: "Rivers"),
    (q: [The longest river \ in Europe?], a: [The Volga]),
  ),
)
```



question page (odd page)



answer page (even page): the columns are mirrored

- The card number appears as “1/3”. It is `none` on blank cards, which is why the `number` function checks it.
- Printing the number on the answer side helps to put the cards back in order and to find a card in the list.

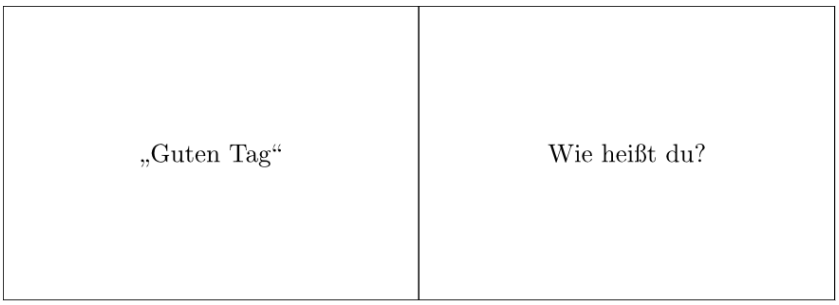
- Slots are designed for a DIN A8 card and grow with larger cards. Their text is `slot-font` / `slot-size` (default 7 pt); `text(size: …)[…]` inside a slot overrides it.
- On small cards there is little room: a long centre slot runs into the corner slots.
- `c.theme` fails on a card without theme; `c.at("theme", default: none)` does not.

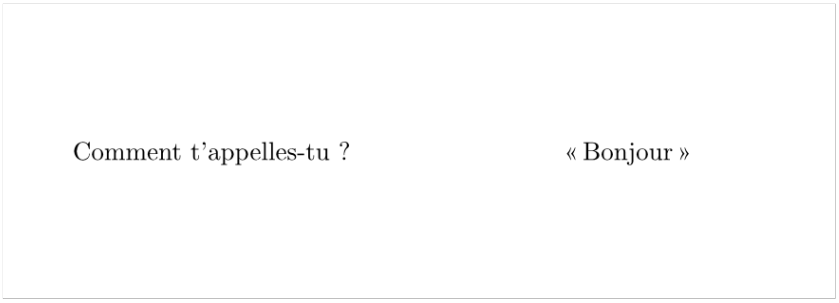
5 Language, fonts and size

```
#import "@preview/flipside:0.2.0": *

#flipside(
  lang: (q: "de", a: "fr"),
  (
    (q: ["Guten Tag"], a: ["Bonjour"]),
    (q: [Wie heißt du?], a: [Comment t'appelles-tu ?]),
  ),
)
```

`lang` sets the language for hyphenation and quotation marks: `"fr"` for both sides, or one per side with `(q: "de", a: "fr")`. Above, the German side gets German quotes „…“ and the French side French ones «…» with the right spaces.





- `font` and `size` set the text of the cards (default: New Computer Modern, 13 pt). The fonts built into Typst are New Computer Modern, Libertinus Serif and DejaVu Sans Mono; with them the cards look the same on every computer.
- The text does **not** grow with the card size. For large cards raise `size`, e.g. `size: 18pt` for DIN A7, `size: 22pt` for DIN A6.

6 Text that is too large

With `fit: true` (the default) a side whose content is too large for the card is shrunk until it fits, so nothing is cut off or runs into the footer.

```
#import "@preview/flipside:0.2.0": *

#flipside((
  (q: [Short question], a: [Short answer]),
  (
    q: [What does photosynthesis need?],
    a: block-left[
      Plants need *light*, *water* and *carbon dioxide*. With the energy of the
      light they make *glucose* and give off *oxygen*:
      $ 6 "CO"_2 + 6 "H"_2"O" -> "C"_6"H"_12"O"_6 + 6 "O"_2 $
      This happens in the *chloroplasts*, mainly in the leaves.
    ],
  ),
))
```

Plants need **light**, **water** and **carbon dioxide**. With the energy of the light they make **glucose** and give off **oxygen**:

$$6CO_2 + 6H_2O \rightarrow C_6H_{12}O_6 + 6O_2$$

This happens in the **chloroplasts**, mainly in the leaves.

Short answer

The answer of card 2 shrunk to fit

The list (Section 10) shows these cards under their number, e.g. `A 84%`: the answer side is printed at 84 %. With `fit: false` nothing is shrunk, and the list marks the cards that are too large in red (“A too large”), so you can shorten them yourself.

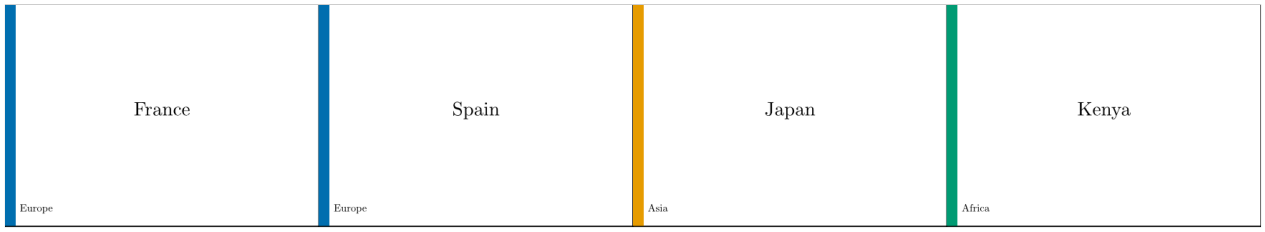
1 F1	Short question	Short answer
2 F1 A 96%	What does photosynthesis need?	Plants need **light**, **water** and **carbon dioxide**. With the energy of the light they make **glucose** and give off **oxygen**: $6CO_2 + 6H_2O \rightarrow C_6H_{12}O_6 + 6O_2$ This happens in the **chloroplasts**, mainly in the leaves.

7 Coloured stripes per theme

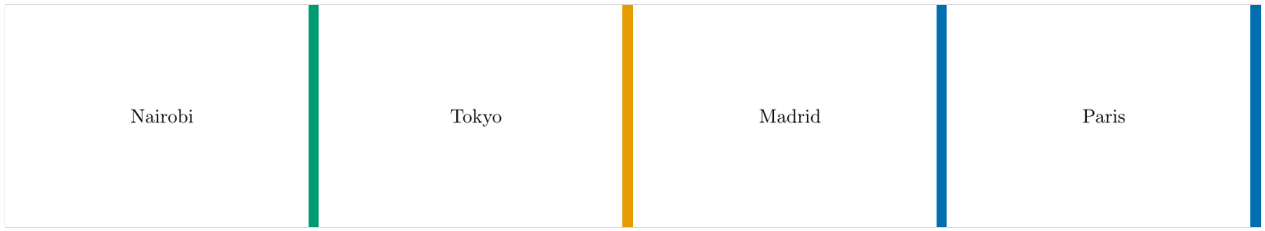
A coloured stripe on one edge of the card makes it easy to sort cut cards. The stripe is printed on the same edge on both sides.

```
#import "@preview/flipside:0.2.0": *

#flipside(
  stripe: auto,
  footer: (left: c => c.theme),
  (
    (theme: "Europe"),
    (q: [France], a: [Paris]),
    (q: [Spain], a: [Madrid]),
    (theme: "Asia"),
    (q: [Japan], a: [Tokyo]),
    (theme: "Africa"),
    (q: [Kenya], a: [Nairobi]),
  ),
)
```



question page (odd page)



answer page (even page): the columns are mirrored

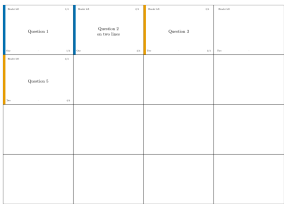
- `stripe: auto`: one colour per theme, in the order the themes appear (a palette that stays distinguishable for colour-blind readers).
- `stripe: ("Europe": blue, "Asia": orange)`: your own colours, by theme; themes not named get no stripe.
- `stripe: c => if c.number > 10 { red }`: any rule, as a function of the card.
- `stripe-width` sets its width (2.5 mm on an A8 card; it grows with the card).

8 Card sizes and paper

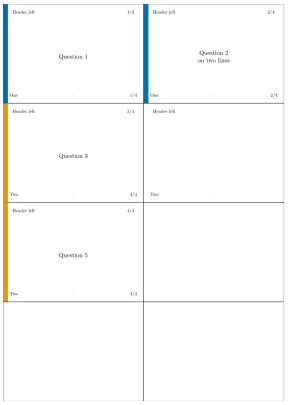
8.1 The sizes

`card:`	Card	Paper	Per sheet	Sheet
`"DIN A8"` (default)	74 × 52 mm	A4	16	landscape
`"DIN A7"`	105 × 74 mm	A4	8	portrait
`"DIN A6"`	148 × 105 mm	A4	4	landscape
`"DIN A5"`	210 × 148 mm	A4	2	portrait
`"US 3x5"`	5 × 3 in	Letter	4	landscape
`"US 4x6"`	6 × 4 in	Letter	2	portrait
`"US business"`	3.5 × 2 in	Letter	10	portrait
`"EU business"`	85 × 54 mm	A4	10	portrait

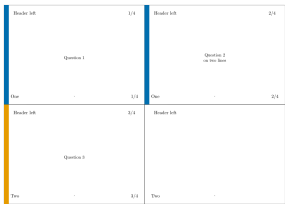
"DIN" can be left out (`"A8"`), and case and spaces do not matter. The DIN sizes fill the A4 sheet exactly. The other sizes are centred; the business cards match the usual **perforated business-card sheets**, so no cutting is needed.



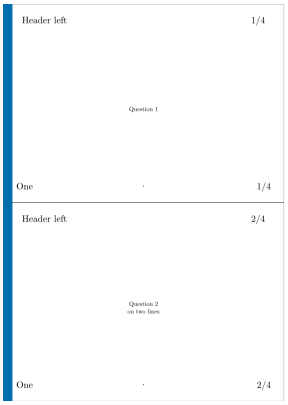
DIN A8



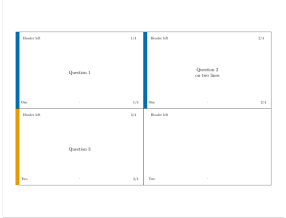
DIN A7



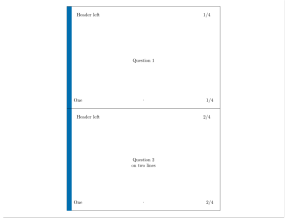
DIN A6



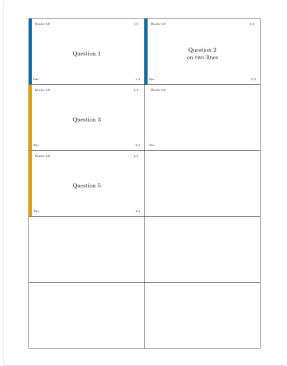
DIN A5



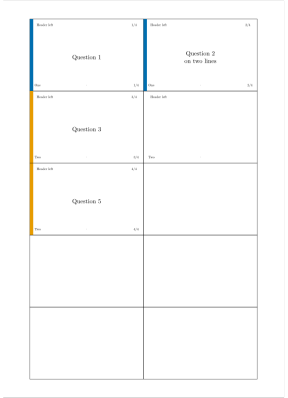
US 3x5



US 4x6



US business



EU business

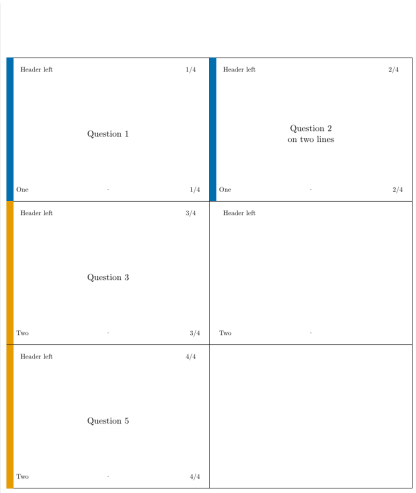
8.2 Paper

The paper follows the card: A4 for DIN and EU sizes, Letter for US sizes. `paper:` chooses another one: `"a4"`, `"us-letter"`, `"a3"`, `"a5"` or `"us-legal"`. The number of cards and the way the sheet is turned are worked out for that paper.

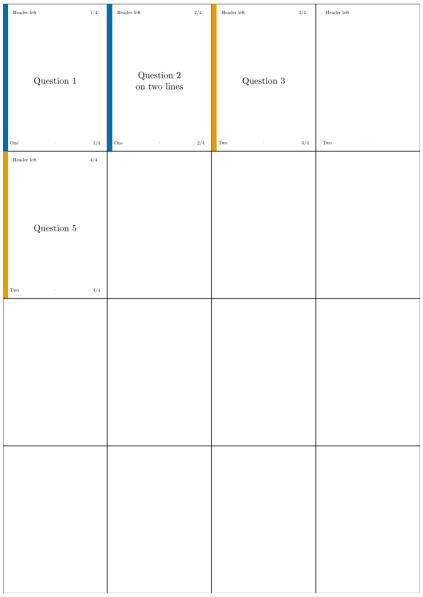
```
#flipside(card: "DIN A7", paper: "us-letter", …)   // 6 per sheet
```

8.3 Upright cards

`orientation: "portrait"` turns any size upright, e.g. 16 upright A8 cards on a portrait sheet (right), or DIN A7 on Letter paper (left, above):



DIN A7 on Letter



DIN A8, `orientation: "portrait"`

8.4 A size of your own

```
#flipside(card: (width: 90mm, height: 60mm), …)
```

As many cards as fit are centred on the sheet. For a perforated sheet you have, give its layout exactly:

```
card: (
  width: 85mm, height: 55mm,       // one card
  cols: 2, rows: 5,                // cards per row and column
  gap: (x: 5mm, y: 0mm),           // space between the cards (or gap: 3mm)
  margin: (left: 15mm, top: 13.5mm), // first card, from the sheet's corner
)
```

The answer page is mirrored, so its left margin is the right margin of the question page; nothing else has to be given.

9 Printing

9.1 Duplex settings

Print **all pages** double-sided, at 100 % (“actual size”):

Sheet	Duplex setting
landscape (DIN A8, A6, US 3x5)	**flip on short edge**
portrait (DIN A7, A5, US 4x6, business cards)	**flip on long edge**

The sheet is turned over like a book page: left and right swap, the top stays at the top.

> Test with the first sheet before printing everything: hold it against the light, the answer behind the top-left card must belong to the top-left question. **Answers upside down?** Choose the other duplex setting. **Answers behind the wrong cards?** Check that the printer is not set to “booklet” or “several pages per sheet”, and that the scale is 100 %.

Printer without duplex: print the odd pages, put the stack back into the tray turned over like a book page, print the even pages. How the paper has to go in depends on the printer: try it with one sheet.

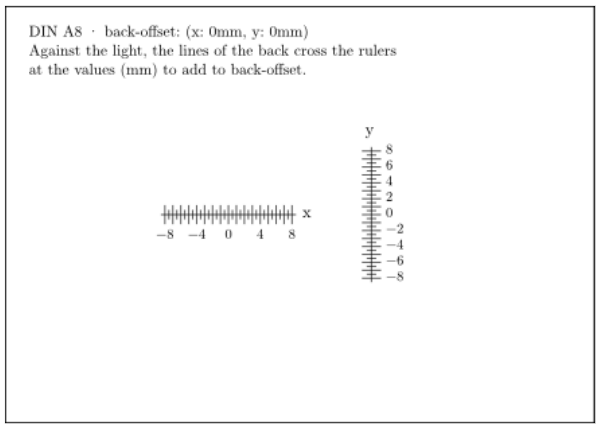
Cutting: cut along the lines of the question page. Printers leave a few mm unprinted at the paper edge, so the outer lines may be missing; that does not matter.

9.2 Calibration: when the back side is shifted

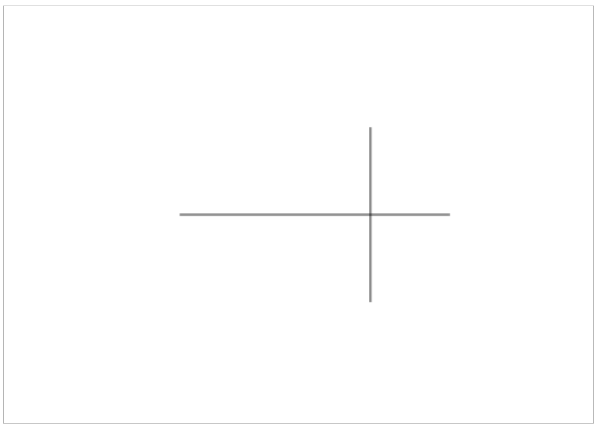
Many duplex printers shift the back side by up to a millimetre or two, so the answers are not centred behind the questions. The **calibration sheet** measures it:

```
typst compile --input layout=calibrate cards.typ calibrate.pdf
```

Print its two pages double-sided at 100 % and hold the sheet against the light. Each card has an **x ruler** and a **y ruler** on the front and two lines on the back; seen through the paper, the lines cross the rulers at the values, in mm, to add to `back-offset`. The front also shows the `back-offset` the sheet was made with.



front: the rulers



back of the same card: the lines

If the vertical back line crosses the x ruler at −0.5 and the horizontal one crosses the y ruler at 1, write:

```
#flipside(back-offset: (x: -0.5mm, y: 1mm), …)
```

or, when compiling, `--input back-offset=-0.5mm,1mm`. Print the calibration sheet again with the new values: the lines should now cross at 0. The values belong to the printer (and its paper tray), not to the deck; keep them in a note or put the line in all your decks for that printer.

9.3 Reprinting some cards

A card got lost, or a few were corrected? `only` prints just these cards, packed onto as few sheets as needed:

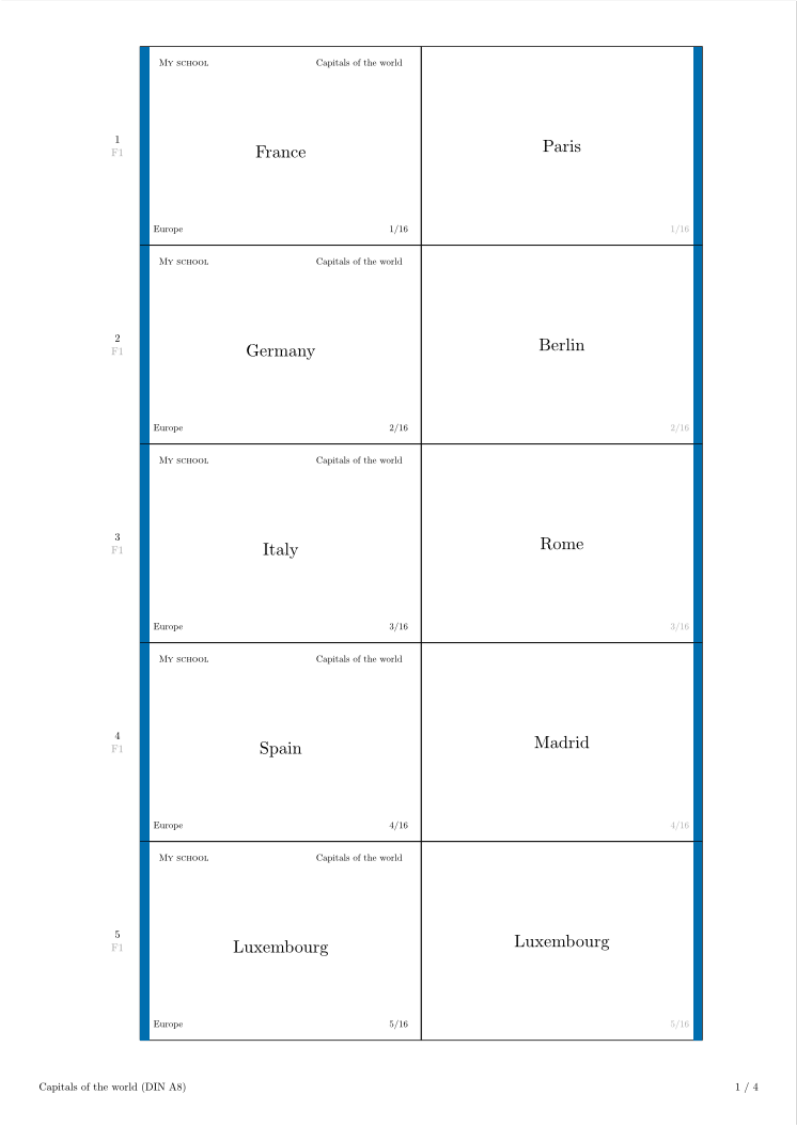
```
typst compile --input only=5-12,17 cards.typ reprint.pdf
```

In the file: `only: "5-12,17"`, `only: (3, 8)`, or a function such as `only: c => c.theme == "Asia"`. The numbers and the total stay those of the whole deck, so the reprinted cards fit back in. It works with the list too.

10 The list: checking the cards

Proof-reading cut cards is tedious. The list shows every card on one row: its number, the sheet it is printed on (F1, F2, …), the question side and the answer side, each exactly as printed.

```
typst compile --input layout=list cards.typ cards-list.pdf
```



- Or in the file: `layout: "list"`. The `--input` always wins over the file, so the file can stay on `"cards"`.
- The list is on the same paper as the cards, landscape for large cards; large cards are shown smaller, so that both sides fit next to each other.
- Cards shrunk to fit, or too large with `fit: false`, are marked under their number (Section 6).
- `title` names the deck in the list's page footer.

11 Command line

Everything that changes between two printouts can be chosen when compiling; it always wins over the file:

Input	Example
`layout`	`--input layout=list`, `--input layout=calibrate`
`card`	`--input card=A7`, `--input card="US 3x5"`
`paper`	`--input paper=us-letter`
`orientation`	`--input orientation=portrait`
`only`	`--input only=5-12,17`
`dir`	`--input dir=both`
`back-offset`	`--input back-offset=0.8mm,-0.5mm` (plain numbers are mm)

A deck file can also take its settings from variables at the top, which is handy in the Typst web app:

```
#let layout = "cards"   // "cards", "list" or "calibrate"
#let card = "DIN A8"

#flipside(layout: layout, card: card, …)
```

12 Reference

```
#flipside(cards, header: (:), footer: (:), back-header: (:), back-footer: (:),
  title: none, lang: "en", font: "New Computer Modern", size: 13pt,
  slot-font: "New Computer Modern", slot-size: 7pt, back-borders: false,
  fit: true, stripe: none, stripe-width: 2.5mm, markup: false,
  back-offset: (x: 0mm, y: 0mm), only: none, layout: "cards",
  card: "A8", paper: auto, orientation: "landscape")
```

Parameter	Default	Meaning
`cards`	—	the cards: `(q: […], a: […])`, entries with fields only, blank cards `(q: [], a: [])`, CSV rows
`header`, `footer`	`(:)`	slots of the question side: `(left:, center:, right:)`, content or `c => …`
`back-header`, `back-footer`	`(:)`	slots of the answer side
`title`	`none`	name of the deck, in the list's page footer
`lang`	`"en"`	`"fr"`, or one per side: `(q: "de", a: "fr")`
`font`, `size`	New Computer Modern, 13 pt	text of the cards
`slot-font`, `slot-size`	New Computer Modern, 7 pt	text of the slots (on an A8 card)
`back-borders`	`false`	outline the answer cards too
`fit`	`true`	shrink a side that is too large
`stripe`	`none`	`auto`, `("Theme": colour)` or `c => colour`
`stripe-width`	`2.5mm`	width of the stripe on an A8 card
`markup`	`false`	read strings (CSV cells) as Typst markup
`back-offset`	`(x: 0mm, y: 0mm)`	shift of the answer pages for the printer
`only`	`none`	only some cards: `"5-12,17"`, `(3, 8)` or `c => bool`
`layout`	`"cards"`	`"cards"`, `"list"`, `"calibrate"` or `"report"` (for the MyCards app)
`card`	`"A8"`	a size name (Section 8) or `(width:, height:, cols:, rows:, gap:, margin:)`
`paper`	`auto`	`"a4"`, `"us-letter"`, `"a3"`, `"a5"`, `"us-legal"`
`orientation`	`"landscape"`	`"portrait"` for upright cards
`dir`	`"forward"`	`"reverse"` or `"both"` (Section 3)
`count`	`n => [(n)]`	what the question shows when more than 2 items are needed

Helpers: `small[…]` (72 %), `tiny[…]` (60 %), `block-left[…]` (left-aligned, justified). **Items, hints and notes** (Section 3): `all`, `any`, `q-only`, `a-only`, `note`, `q-note`, `a-note`. **Values:** `card-sizes` (the size table), `papers`, `stripe-palette` (the colours of `stripe: auto`).

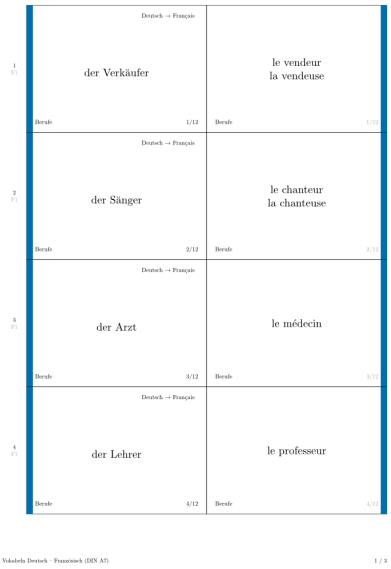
13 Examples

The folder `examples` of the package holds complete decks. Compile them from the package folder with `typst compile --root . examples/<name>.typ`.

13.1 Vocabulary from a CSV file

German–French words read from `vocabulary.csv`, DIN A7 cards (8 per sheet, text at 18 pt), a stripe per theme, German hyphenation and quotes on the question side, French on the answer side.



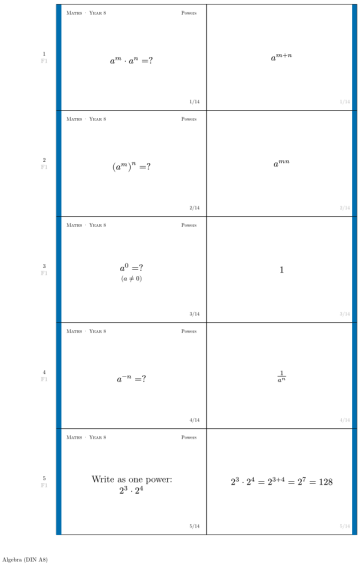


`examples/vocabulary.typ`: first sheet, and the list

13.2 Maths

Formulas, several lines per side, lists and small print; a stripe colour chosen per theme; the theme in the header; a long answer that is shrunk to fit.



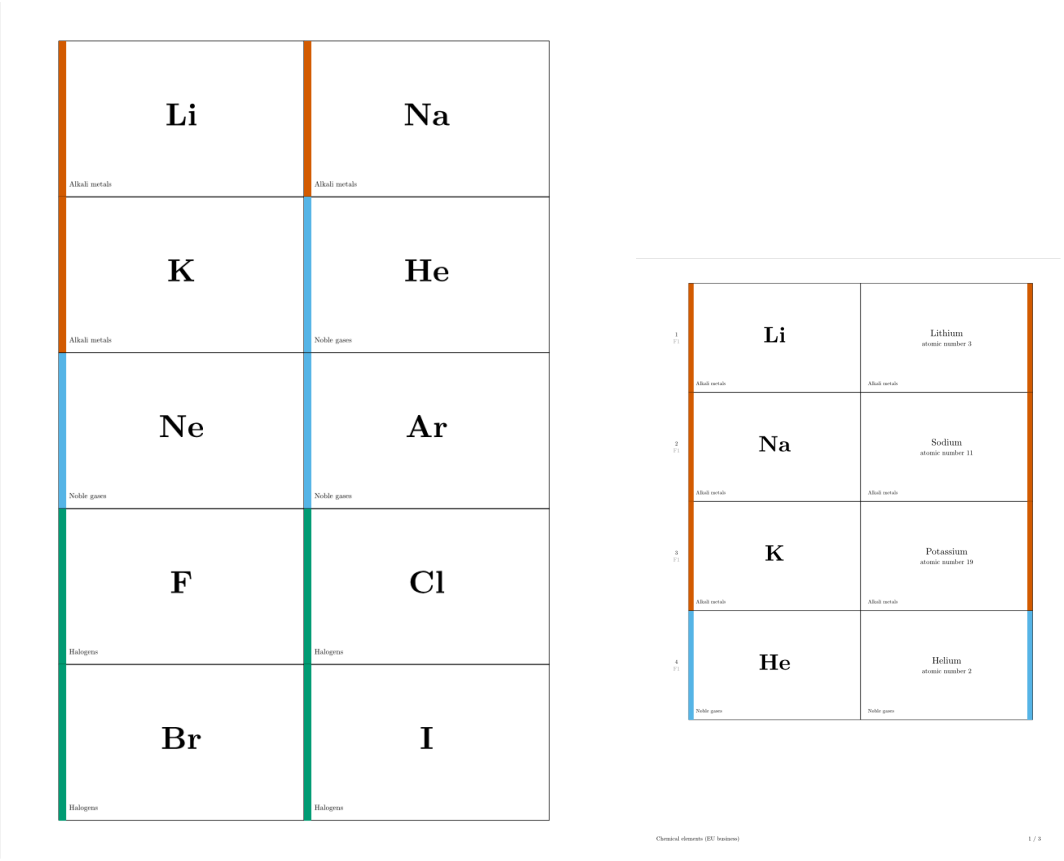


Algebra (DIN A8) 1 / 3

`examples/maths.typ`: first sheet, and the list

13.3 Business-card sheets

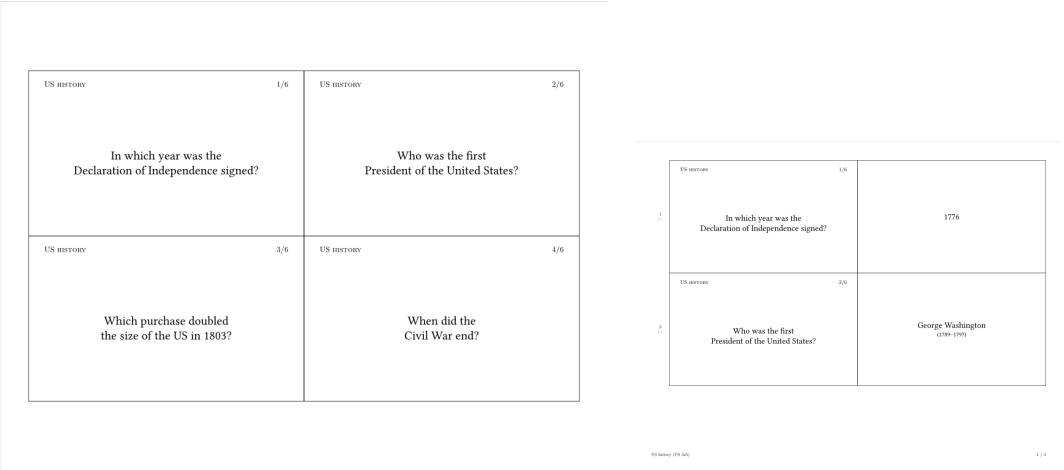
Chemical elements on perforated 85 × 54 mm sheets (10 per A4): no cutting. The cards are made by a function `element(symbol, name, number)`.



`examples/business-cards.typ`: first sheet, and the list

13.4 US index cards

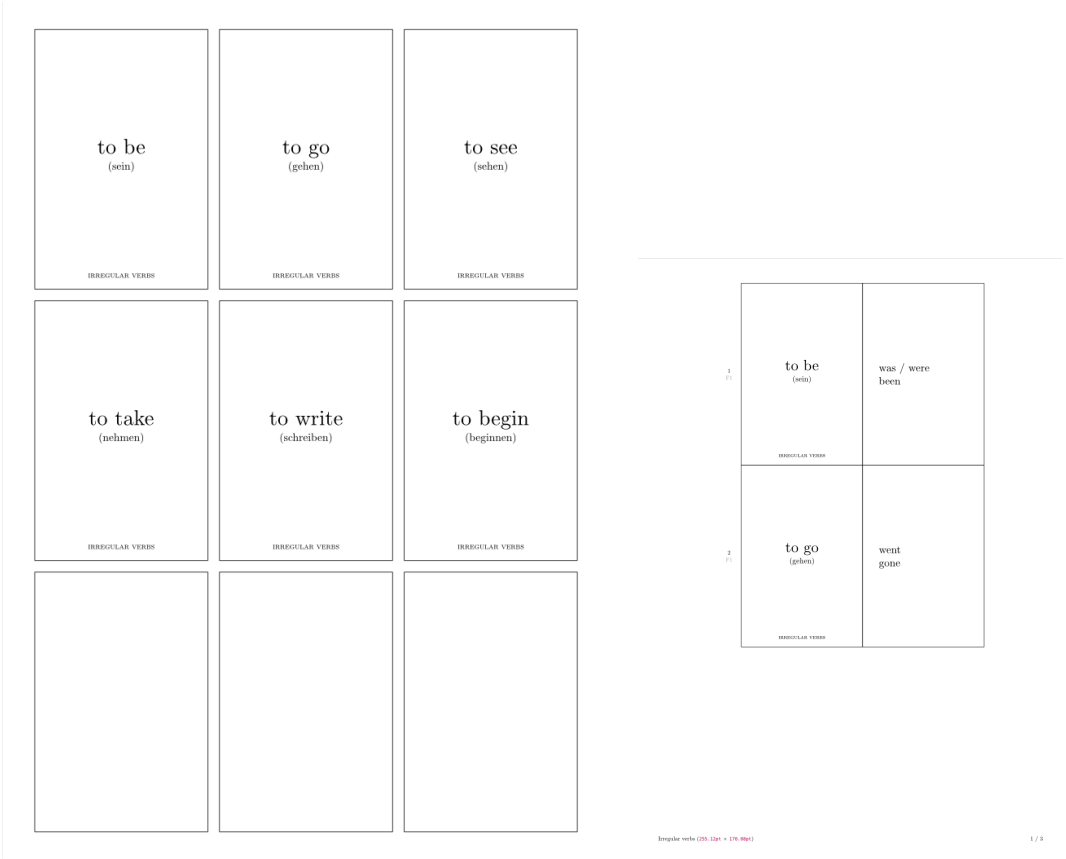
3 × 5 inch cards on Letter paper, in Libertinus Serif.



`examples/us-index-cards.typ`: first sheet, and the list

13.5 Upright cards of your own size

60 × 90 mm upright cards, 3 × 3 per sheet with gaps between them, made by a function: irregular verbs.



`examples/upright-cards.typ`: first sheet, and the list

14 Questions and answers

The answers are behind the wrong cards Make sure you print all pages at 100 %, double-sided, with the duplex setting of Section 9, and not as a booklet. Test with one sheet first.

The answers are all a little off-centre The printer shifts the back side. Measure it with the calibration sheet and set `back-offset`.

Some text is very small on a card It did not fit and was shrunk (`fit`). The list shows which cards; shorten them, or use a larger card.

The text looks small on large cards The text does not grow with the card; raise `size`.

I want different headers for some cards Put a field in an entry (e.g. `(chapter: "2"),`) and show it with a function slot: `c => c.chapter`.

How do I print the same deck in two sizes? Keep the file and choose the size when compiling: `--input card=A7`.

Can I use my own fonts? Yes, any font installed on your computer: `font: "Arial"`. The built-in fonts look the same everywhere, others only where they are installed.